

* ISA serves as boundary between hardware & software

- Addressing mode
- Memory addressing
- Operations
- Control flow
- Encoding

* pipelining $\rightarrow$ assembly of instructions /
overlap of execution

* Latency $\rightarrow$ Time taken to complete the task

* Throughput $\rightarrow$ $ET \propto \frac{1}{P}$

If Technique 2 is n times faster than Technique 1

$$n = \frac{\text{Execution time of Technique 1}}{\text{Execution time of Technique 2}} \\ = \frac{\text{Performance of Technique 2}}{\text{Performance of Technique 1}}$$

* CPU Time = Instruction Count (IC) $\times$ Clock cycle
 $\times$ CPI

$\downarrow$
cycles per instruction
Reducing them increases performance &
reduces execution time

Fetch $\rightarrow$ Decode $\rightarrow$ Execute $\rightarrow$ Memory $\rightarrow$ Write back

IF	ID	EXE	MEM	WB
mem	reg	alu	mem	reg

ARM7 $\rightarrow$ 3 Stages

- FETCH (F)
- DECODE (D)
- EXECUTE (E)

* Pipeline can have many instructions at a phase but all of them are different
* Can only fetch one instruction at a given phase

$k \rightarrow$ no. of stages
 $t_c \rightarrow$ clock cycle time
 $n \rightarrow$ no. of instruction

Execution Time for pipeline

$$= (k \times t_c \times 1) + (n-1) \times t_c$$

Execution Time for non-pipeline

$$= n \times k \times t_c = n \times t_p$$

$$\text{Speed up} = \frac{\text{ET of non-pipeline}}{\text{ET of pipeline}}$$

$$= \frac{n \times t_p}{(k + (n-1)) t_c} = \frac{n \times k \times t_c}{(k + (n-1)) t_c} \approx k$$

Q. Compute the Execution Time on 5 stage processor: Number of instruction = 100
Cycle Time = 60

$$\text{Pipeline ET} = (5 \times 60 \times 1) + (99 \times 60) \\ = 6240$$

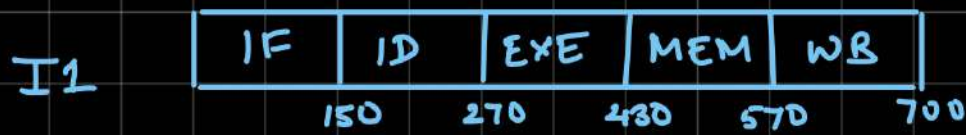
$$\text{Non-pipeline ET} = 5 \times 60 \times 100 \\ = 30000$$

$$\text{Speedup} = \frac{30000}{6240} = 4.8$$

Q.

IF	ID	EXE	MEM	WB
300	400	350	550	100
200	150	100	190	140

Q. Draw the timing diagram for 3 instructions
with instruction fetch = 150, 120, 160, 140, 130
ID EXE MEM WB



Q. A 5 stage pipeline processor has stage delays as
150, 120, 160, 140 and 130 ns

What is the time taken to execute 100 instructions?

What is the speed up of pipeline processor?



↳ pipelined register

delay caused by the pipeline register → pipeline overhead

max. delay b/w
the clock arrival
at any 2 registers

also due to
clock skew &
pipeline delay

$$\text{Speedup} = \frac{\text{Total time} \times n}{[K + (n-1)] \text{ (Highest time + overhead)}}$$

Q. A 5 stage pipeline processor has stage delays as 150, 120, 160, 140 and 130ns. The register overhead is 5ns each

What is the time taken to execute 100 instruc.

What is the speedup of pipeline processor?

$$\text{Execute of nonpipe} = (150 + 120 + 160 + 140 + 130) \times 100$$

$$\text{Time} = 70000$$

$$\text{ET of pipe} = [5 + (100 - 1)] \times (160 + 5) = 17160$$

$$n = \frac{\text{ET of non-pipe}}{\text{ET of pipe}} = \frac{(150 + 120 + 160 + 140 + 130) \times 100}{(5 + 99) 165}$$

$$= 4.079$$

DECODE PHASE →

Sign extension is done in the decode phase
↳ checking for the condition of branch equality

MEMORY PHASE →

Effective address calculation is done in the memory phase

HAZARDS

Structural Hazard: Due to the structure of pipeline, resource conflict

Data Hazard: data dependency

Inst. depends on the source of the prev. instr. still in pipeline

Control Hazard: tries to change the PC

STALLS: "bubble"

common solution for any type of hazard

whenever there's a stall in the pipeline, no new instructions are followed

The entire pipeline stalls in the remaining instruction

Q. Draw the pipeline diagram for the following instruction:

add R1, R2, R3

add R4, R5, R6

sub R7, R4, R8

mul R9, R10, R11

I ₁	IF	ID	EXE	MEM	WB					
I ₂		IF	ID	EXE	MEM	WB				
I ₃			IF	ID	stall	stall	EXE	MEM	WB	
I ₄				IF	stall	stall	ID	EXE	MEM	WB

WAYS TO SOLVE HAZARDS

UNIFIED MEMORY

IF = MEM → unified memory
and at a certain time will create a hazard

- ① Stall
 - ② Split cache memory
- } solution

Eliminating any steps → steps
WB & WB

↳ follow phases in pipeline

ID & WB → stall

↳ 1/2 clk cycle, RW, 1/2-RR

↓
register
write

↓
register
read

Q. Which combinations of the instructions below are involved in a structural hazard when handled by a 5-stage processor pipeline?

- a. Instruction 1: add R2, R3, R4
- b. 2: ldr R5, [R6]
- c. 3: sub R7, R8, R9
- d. 4: sub R10, R11, R1
- e. 5: str R2, [R12]

IF	ID	EX	MEM	WB																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																														
----	----	----	-----	----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

DISADVANTAGES

- Stall cycle is waste cycle
- Adds to pipeline delay & increases $\frac{CPI}{\downarrow}$
($CPI > 1$) Ideal = 1

$$\text{Speedup} = \frac{\text{Avg. ET of non-pipe}}{\text{Avg ET of pipe}}$$

$$= \frac{(IC \times CPI \times CT)_{\text{non-pipe}}}{(IC \times CPI \times CT)_{\text{pipe}}} = \frac{CPI_{\text{unpipe}}}{CPI_{\text{pipe}}}$$

$$= \frac{K}{1 + \text{pipeline stalls}} \rightarrow \text{pipeline depth}$$

If no pipeline stalls are there, speedup is equal to pipeline depth

To reduce hazard (Mem and fetch is same phase)

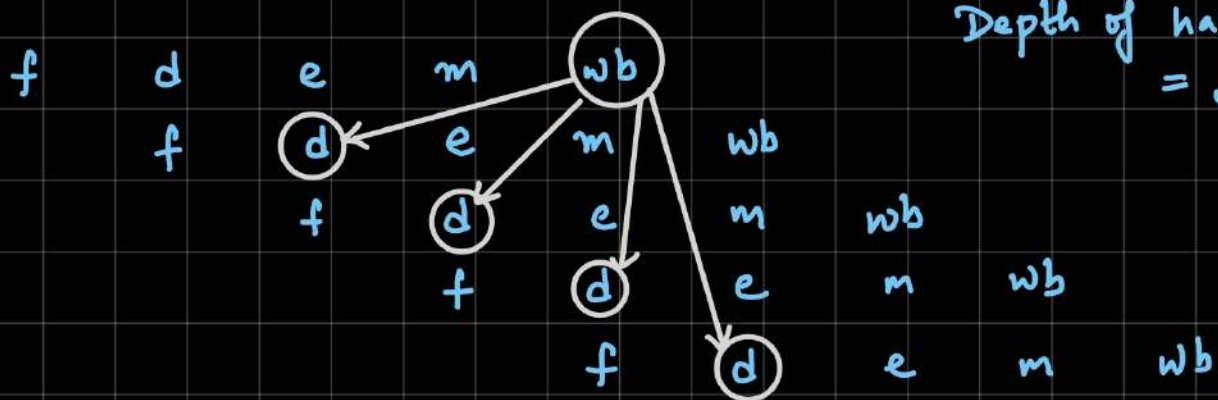
→ Add stalls

→ Add split cache

unified memory → structural hazard

Find the depth of the hazard for the following instructions

add R1, R2, R3
sub R4, R1, R3
and R6, R1, R7
xor R10, R1, R11
or R8, R1, R11



DATA DEPENDENCIES

RAW (Read after Write)

↳ true dependency

I: add R1, R2, R3
J: sub R4, R1, R3 } hazard } define-use dependency

Cannot read J until I is completed

ldr R1, a
add R2, R1, R1 } load-use dependency

WAR (Write after Read)

↳ anti-dependency

I: sub R4, R1, R3
J: add R1, R2, R3
K: mul R6, R1, R7

Instr writes
operand before
instr reads it

WAW (Write after write)

↳ output dependence

out of order → wrong results

sub R1, R4, R3
add R1, R2, R3

Instr writes
operand before
instr writes it

* To solve false dependencies → rename the register

Q. Identify the dependencies in the given set of instr.

- ① add R3, R1, R2
- ② sub R4, R3, R5
- ③ mul R6, R3, R7
- ④ and R8, R4, R6
- ⑤ orr R6, R10, R11

TRUE: ① & ③ ^{R3}, ② & ④ ^{R4}, ① & ② ^{R3}, ③ & ④ ^{R6}

ANTI: ④ & ⑤ R6

OUTPUT: ③ & ⑤ R6

} no hazard.

ATTACKING DATA HAZARD

In Software

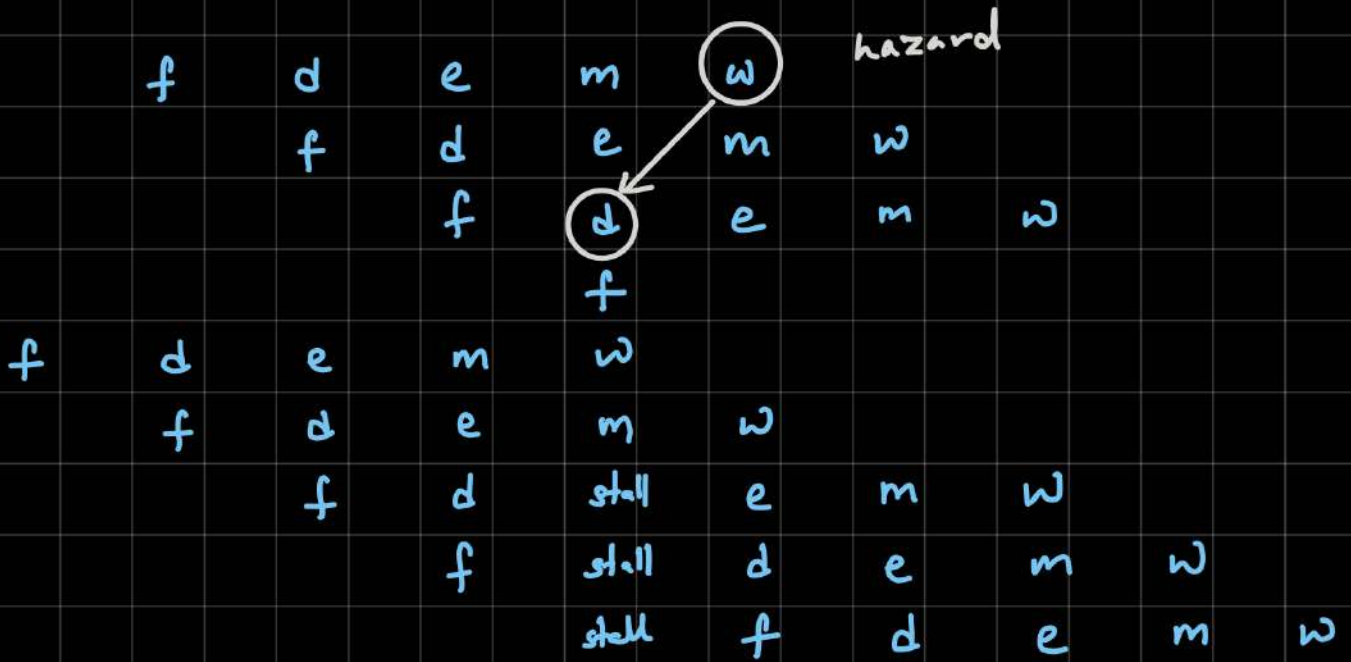
- Solution 1: Re-order instruction
- Solution 2: Insert independent or no-ops instr.

In Hardware

- Solution 1: insert bubbles or stalls
- Solution 2: Data Forwarding

Q. Find whether the following set of instruction require a stall in the pipeline

ADD R0, R0, R1
 ADD R2, R3, R4
 ADD R5, R0, R4
 SUB R6, R4, R7
 ADD R8, R4, R7



Rearranging the instructions instead of stalling

① ADD R0, R0, R1
 ② ADD R2, R3, R4
 ④ SUB R6, R4, R7
 ⑤ ADD R8, R4, R7
 ③ ADD R5, R0, R4

} if no partitioning

① ADD R0, R0, R1
 ② ADD R2, R3, R4
 ④ SUB R6, R4, R7
 ③ ADD R5, R0, R4
 ⑤ ADD R8, R4, R7

} partitioning

Q. Find the no. of no-ops to be inserted for the following set of instructions

SUB R2, R1, R3

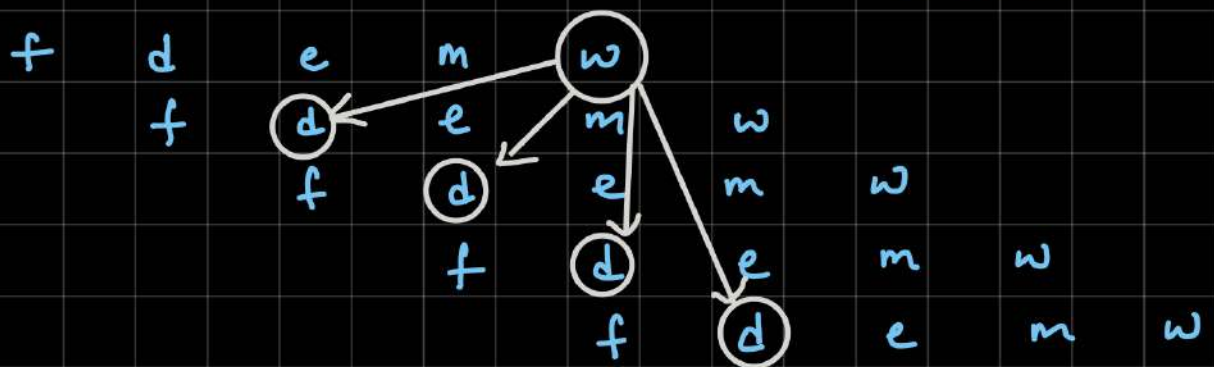
AND R12, R2, R5

ORR R10, R6, R2

ADD R4, R2, R2

STR R5, (R2, #100)

2 hazards
↓
2 stalls

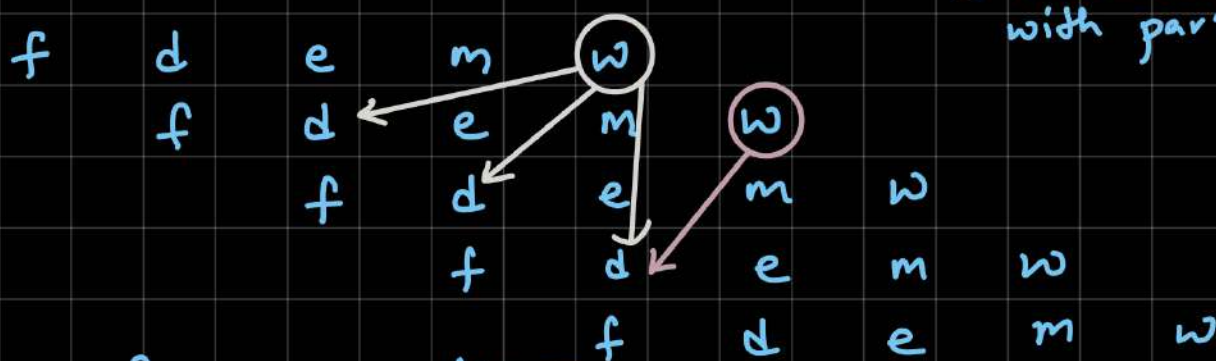


2 no-ops to be inserted after ①

SUB	R2, R1, R3		
MOV	R0, R0	/	NOP
MOV	R0, R0	/	NOP
AND	R12, R2, R5		
ORR	R10, R6, R2		
ADD	R4, R2, R2		
STR	R5, (R2, #100)		

Q. How many no-ops cycle are needed?

SUB R2, R1, R3
 AND R4, R2, R5
 ORR R8, R2, R6
 ADD R9, R4, R2
 RSB R1, R6, R7



3 hazards
with partitioning

2 no-ops after ①
 1 no-op after ② and before ④

SUB R2, R1, R3
 NOP
 NOP
 AND R4, R2, R5
 ORR R8, R2, R6
 NOP
 ADD R9, R4, R2
 RSB R1, R6, R7

Q. If reordering is used, how many no-ops is used?

ADD R3, R1, R2
SUB R4, R3, R5
MUL R6, R3, R7
AND R8, R4, R6



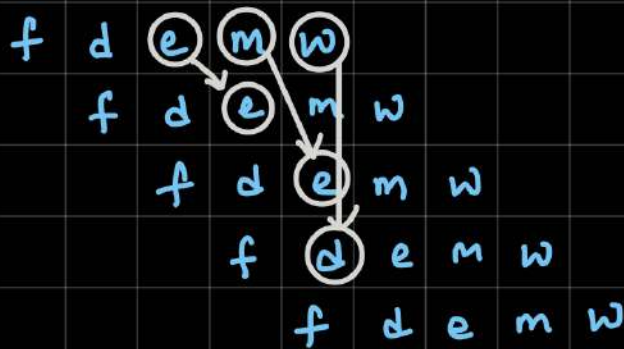
4 hazards

ADD R3, R1, R2
NOP
NOP
SUB R4, R3, R5
MUL R6, R3, R7
NOP
NOP
AND R8, R4, R6

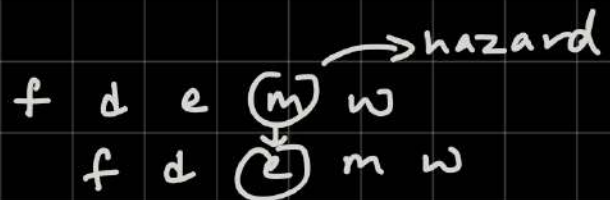
No reordering
is possible

Q. Find the data forwarding path for the following instruction
\$ → register ✓

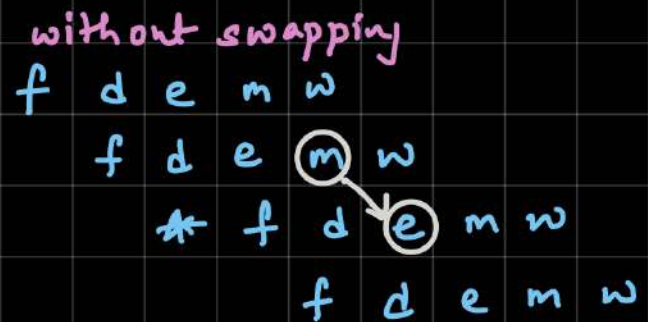
DADD R1, R2, R3
DSUB R4, R1, R5
AND R6, R1, R7
OR R8, R1, R9
XOR R10, R1, R11



LDR R1, [R2]
DSUB R4, R1, R5



Q. LDR R7, [R2]
LDR R6, [R2, #4]
ADD R4, R5, R6 ← NOP
STR R6, [R2, #4]



1 NOP f (2) → (3) data forwarding
m → e

with swapping

LDR R6, [R2, #4]
LDR R7, [R2]
ADD R4, R5, R6
STR R6, [R2, #4]

with data forwarding



without data forwarding



Q. $a = b + c$ and $d = e - f$. Identity dependencies.

LDR R2, =b

LDR R3, =c

ADD R1, R2, R3

STR R1, =a

LDR R2, =e

LDR R3, =f

SUB R1, R2, R3

STR R1, =d

CONTROL HAZARDS

Flush instruction → remove everything from pipeline to reach label at branching

Instead add stall still execution phase

- All branches require 2 stalls

To reduce stall - find branch is T or F at decode

- All branch require 1 stall

But it can't be moved to fetch because otherwise fetch clock cycles will increase

Alloc

MAX

Avoid

Q. With stalls, pipeline has 5 stages and each stage requires 1 clock cycle

Q.

f	d	e	m	w	
	f	d	e	m	w

L1 : LDR R1, [R4]

LDR R2, [R4, #400]

ADD R3, R1, R2

STR R3, [R4]

SUB R4, R4, #4

BNEZ R4, L1

L1 : LDR R1, [R4]

} 100 times

f d e m w

f d e m w

f d * e m w

f * d e m w

f d e m w

f d * e m w

ends here

Clock cycles = 9

$$\text{Speedup} = \frac{3005}{901} =$$

Q.

L1 : LDR R1, [R4]

LDR R2, [R4, #400]

SUB R4, R4, #4

ADD R3, R1, R2

STR R3, [R4]

BNEZ R4, L1

L1 : LDR R1, [R4]

f	d	e	m	w					
	f	d	e	m	w				
		f	d	e	m	w			
			f	d	e	m	w		
				f	d	e	m	w	
					f	d	e	m	w

clock cycles = 7

$$\text{Speedup} = \frac{3005}{701} = 4.2867$$